

22-Project Full-Stack Developer Roadmap

Anup Bhandari | JavaScript | Project-first | No 30-day deadline

Purpose. Finish 22 portfolio projects selected from the supplied cohort assignment ideas and visible video project cards. Start with React and Node, then build increasingly complete full-stack systems. The timeline is an estimate, not a deadline. Complete one project before moving to the next.

Important scope note. Assignment titles are grounded in the supplied materials. Feature lists here are a proposed implementation plan because some video-card rubrics were unreadable. Before a cohort submission, compare its full official rubric and adjust the checklist.

How to work and when to advance

- Use JavaScript, React + Vite, Node + Express, PostgreSQL + Prisma, Tailwind/CSS and npm. Next.js can replace React + Express in a later capstone if the assignment allows.
- Each project: define user stories and data flow; implement core features; test success and failure paths; add responsive layout when it has a UI; document and demonstrate.
- Each repo: descriptive README, screenshots, setup commands, .env.example, sample data or demo account, API docs if applicable, short demo clip, deployed link when practical. Never commit secrets.
- A project is complete when a fresh clone can run with its README and the completion check below works. Avoid treating tutorial copy/paste as independent skill.
- Estimated effort: 3-6 hours for smaller React projects, 8-15 hours for backend projects, 20-45 hours for larger full-stack projects. Total roughly 250-450 focused hours across 4-8 months at a sustainable pace.

Stages

Stage	Projects	Estimated effort	Primary evidence
React	01-05	25-45 h	UI, API use, state
Node + DB	06-11	55-90 h	HTTP, SQL, auth
Full-stack	12-19	130-230 h	End-to-end products
Advanced	20-22	40-85 h	Realtime, architecture, deployment

Stage 1 - React fundamentals through finished apps

01. Movie Search App (2-3 days)

Build: Search external API | debounce and loading/error/empty states | detail view and responsive cards

Completion check: Search and detail work; network failure and no-result cases are handled.

Portfolio evidence: Screenshots, demo link, API configuration guide.

02. E-commerce Product Listing with Pagination (2-4 days)

Build: Cards, category/price filters and sort | pagination or URL query state | cart using Context or Zustand

Completion check: Filter, sort, page and cart totals remain correct together.

Portfolio evidence: Screen recording covering filters and cart; component diagram.

03. Custom Form with Dynamic Inputs and Validation (2-3 days)

Build: Schema-driven fields and conditional questions | required/email/password validation | review screen and accessible errors

Completion check: Invalid input cannot submit; changing a condition updates relevant fields.

Portfolio evidence: Field schema and validation examples in README.

04. Social Media Feed with Infinite Scroll (2-4 days)

Build: Paginated fetch | IntersectionObserver | retry, deduplication, end-of-feed | like/bookmark local state

Completion check: Each page loads once; retries do not duplicate posts.

Portfolio evidence: Demo of scroll, error/retry and end state.

05. Blog Dashboard with React and Firebase (3-5 days)

Build: Login | create/edit/delete posts | draft/published state | search and responsive dashboard

Completion check: Each signed-in author manages their own posts; refresh preserves data.

Portfolio evidence: Auth and CRUD demo; security-rule explanation.

Stage 2 - Node, APIs, SQL and security

06. Simple REST API (2-3 days)

Build: Express routes for item CRUD | request validation | central error handler | HTTP status codes

Completion check: Postman collection passes create/list/update/delete plus 400/404 cases.

Portfolio evidence: Endpoint table and request collection.

07. URL Shortener (3-4 days)

Build: Validate target URLs | unique short codes | PostgreSQL persistence | redirect and click counter

Completion check: Codes survive restart; redirects work; unknown code gives 404.

Portfolio evidence: Schema and curl redirect example.

08. Blog API with Database Integration (3-5 days)

Build: Post/category schema | CRUD and migrations | pagination/search | foreign-key and validation errors

Completion check: API operates on SQL data and pagination has stable results.

Portfolio evidence: ER diagram, migration/setup and request collection.

09. User Authentication with JWT (3-5 days)

Build: Register/login | bcrypt hash | protected profile endpoint | token expiration and auth failures

Completion check: Plaintext passwords are absent; protected endpoint rejects missing/invalid token.

Portfolio evidence: Threat notes and sanitized auth flow diagram.

10. File Upload with Node.js (2-4 days)

Build: Multipart endpoint | type/size checks | store file and metadata | download/delete only with permission

Completion check: Wrong type and oversize fail; authorized user can retrieve own file.

Portfolio evidence: API sample and upload constraints.

11. Task Management API with RBAC (4-6 days)

Build: User/task/role schema | assignments and status | admin/member authorization | filter and pagination

Completion check: Member cannot edit another user task; admin actions work as documented.

Portfolio evidence: Role-permission matrix and automated ownership checks.

Stage 3 - Full-stack systems: frontend, API and database

12. DevBoard - Task Management System (5-7 days)

Build: React auth screens and task dashboard | connect task API to PostgreSQL | create/edit/status/due dates | per-user ownership and filters

Completion check: Two accounts see separate data and tasks persist after reload.

Portfolio evidence: Deployed link, role demo, architecture and screenshots.

13. Inkwell - Blog Publishing with Admin Approval (5-8 days)

Build: Writer drafts and submit | admin approve/reject queue | public approved posts | role checks and editing rules

Completion check: Writer cannot approve; unapproved content stays private.

Portfolio evidence: Two-account walkthrough and state diagram.

14. HireMe - Job Board (7-10 days)

Build: Employer job CRUD | applicant profile, search and apply | one application per job | employer application review and statuses

Completion check: Employer posts; applicant applies; employer reviews; permissions hold.

Portfolio evidence: ER diagram, demo accounts and 1-minute video.

15. Review Management Tool (4-6 days)

Build: Business profile and review form | moderation/status | rating summary and search | owner dashboard

Completion check: Visitors submit once as designed; owner can moderate without modifying other businesses.

Portfolio evidence: Screenshots and review lifecycle explanation.

16. Custom Form Builder (6-9 days)

Build: Drag/drop or ordered fields | field types and conditional logic | public share link | submissions dashboard and CSV export

Completion check: Creator publishes form; respondent submits; owner sees correct saved responses.

Portfolio evidence: Builder-to-submission demo and field schema.

17. Quick Commerce Platform (8-12 days)

Build: Catalog and stock | cart and server-side order total | checkout with mock payment | admin inventory/order view

Completion check: Out-of-stock purchase is blocked; order total is calculated on server.

Portfolio evidence: Buyer/admin demos and clearly labeled mock payment.

18. Referral Program Platform (5-8 days)

Build: Referral codes and signup attribution | duplicate/self-referral rules | reward ledger | admin reporting

Completion check: A valid referral is counted once; invalid/self referrals earn nothing.

Portfolio evidence: Rule table and test scenarios.

19. CampusHub - Role-Based College Portal (8-12 days)

Build: Student/teacher/admin roles | course/notice management | enrollments and submissions | role-protected views

Completion check: Each role can complete its flow without accessing disallowed data.

Portfolio evidence: Permission matrix, ER diagram and seeded users.

Stage 4 - Advanced projects and production practice

20. Chat Application (6-9 days)

Build: Socket.io private conversations | persist message history | reconnect and unread states | authorization on rooms

Completion check: Two sessions exchange messages; outsider cannot join conversation.

Portfolio evidence: Parallel-browser demo and socket event notes.

21. Team Builder SaaS (7-11 days)

Build: Workspace invite flow | members and roles | team projects or boards | audit trail and subscription mock

Completion check: Invitation joins correct workspace; tenant data never leaks across teams.

Portfolio evidence: Multi-team demo and tenancy diagram.

22. Dockerized Node API + CI/CD Deployment (5-8 days)

Build: Dockerfile and compose with PostgreSQL | health endpoint and migrations | GitHub Actions build/test | deployment, logs and rollback notes

Completion check: Fresh setup starts via documented commands; build/tests run in CI; production health is visible.

Portfolio evidence: Compose file, pipeline badge, deployment README and running URL.

Skill checks after each stage

- After 01-05: build a React screen from a blank project, explain props/state/effects, call an API and handle failures without following a video.
- After 06-11: design a relational schema, write and protect CRUD endpoints, explain status codes and reproduce errors using Postman.
- After 12-19: take a small client requirement and independently deliver frontend, backend, schema, validation, access rules and a working deployment.
- After 20-22: diagnose broken deployments, explain basic security and scaling choices, and show evidence in clean repositories.

How to present this on GitHub

Pin 4-6 strongest repositories: DevBoard, Inkwell, HireMe, CampusHub and one advanced project. Keep all 22 in a portfolio index with one-line problem, your contribution, stack and link. Quality and your ability to explain tradeoffs matter more than the raw count. It is fair to call yourself an aspiring or junior full-stack developer while building; use "full-stack developer" confidently when you can independently deliver and maintain the complete app, including deployment and access control.

Prompt to hand this PDF to another GPT

"I am Anup, learning full-stack JavaScript. This PDF is my active project plan. We are at project number __, milestone __. First inspect the relevant project scope and completion check. Ask for my current code or what works now. Give one small next task with a simple data-flow diagram and Nepali explanation. Let me try coding before showing full code; review my code and test the core flow. Keep the project portfolio-ready. The project titles originate in my cohort materials, while the feature scopes are planning proposals; obey any official rubric I provide."

Personal progress log template

Project # | date started | date completed | repo URL | demo URL | core flow tested | failure cases tested | README/screenshots | one concept learned | one bug fixed.